

ZollPilot

Vollständigkeits- und Konsistenzprüfung von Zoll- und Versaniddokumenten, von der Akte bis zur Nachforderung

Vor einer Ausfuhr liegt ein Stapel Belege: Handelsrechnung, Packliste, Bill of Lading, Ursprungserklärung, Ausfuhrbegleitdokument. Ein Mensch prüft, ob das zusammenpasst. ZollPilot führt dafür pro Sendung eine Akte, liest die Belege, prüft sie gegen ein Regelwerk, fordert nach, was fehlt, und lässt einen Menschen übersteuern, ohne den Befund zu ändern. n8n orchestriert, deterministischer Code entscheidet, kein Modell in der Entscheidung. Konzept, Fachrecherche, Workflows, Extraktion, Regelwerk, Oberfläche, Betrieb und Dokumentation in einer Hand.

7

n8n-Workflows, jeder als Export im Repo und als Bild aus dem Editor

14

ausführbare Prüfredeln, jede mit Rechtsgrundlage und Test, aus einem Fachkatalog von 52

3

Leser hinter einer Nahtstelle: Tesseract, Google Document AI, Azure, alle 570 von 570 Feldern

10

Container in einem Compose: n8n, Postgres, Extraktion, nginx, Mail, Monitoring

11

Architekturentscheidungen als ADR, jede mit dem Abschnitt „Wann wir anders entscheiden würden“

415

Tests in vier Sprachen, acht CI-Jobs, Rauchtest in neun Runden gegen den laufenden Stack

Gabriel Suter · Einzelumsetzung · Konzept, Fachrecherche, Workflows, Extraktion, Regelwerk, Oberfläche, Betrieb, Dokumentation

n8n 1.114 · Node 24 · Python 3.12 · Angular 22 · Postgres 16 · Docker Compose · Prometheus, Grafana · GitHub Actions · September 2026

kontakt@gabrielsuter.dev · gabrielsuter.dev · Prototyp zu einem Anforderungsprofil des Marktes: n8n-Workflows für Dokumentenprozesse in Logistik und Zoll

1 Kurzfassung

ZollPilot prüft vor einer Ausfuhr, ob die Belege einer Sendung vollständig sind und zusammenpassen. Belege kommen als PDF, Scan oder XML-Datensatz herein, werden gelesen und als Behauptungen an eine Akte gehängt. Ein Regelwerk entscheidet auf den daraus abgeleiteten Fakten über Freigabe oder Blockade, fehlende Nachweise werden per Mail nachgefordert, und eine Antwort mit Anhang findet ihre Akte über das Aktenzeichen wieder. Eine Sachbearbeitung sieht jeden Befund mit Fundstelle und kann ihn übersteuern, mit Name und Begründung.

Der Kern in einem Satz

Belege sind Behauptungen, die Akte hält den geprüften Zustand: Modelle und OCR lesen, deterministischer Code in `src/` entscheidet, und ein Mensch übersteuert, ohne den Befund zu ändern.

2 Ausgangslage und Ziel

Der Auftraggeber dieses Projekts ist ein Anforderungsprofil, wie es der Markt für n8n-Projekte in Logistik und Zoll ausschreibt: Workflows mit IDP und OCR, Vollständigkeitsprüfung, Nachforderungsprozess, Integrationen, Monitoring und Prozessdokumentation. Ziel war nicht ein Demo-Workflow, sondern ein Repo, in dem jeder Punkt des Profils einen Beleg hat oder ehrlich als nicht belegbar dasteht. Das Anforderungsmapping steht im Repo und wird von einem Gate geprüft: Jeder Pfad in der Belegspalte muss existieren.

Aus der Fachrecherche zu Dokumententypen, Pflichtangaben und Zollprozess entstanden vier Regeln, die jede Entscheidung im Projekt gebunden haben: Nachweis statt Dokument, Rohwert und Aktenwert getrennt, Regeln sind Daten, kein Modell in der Entscheidung. Alles, was diesen Regeln widerspricht, ist eine ADR wert oder nicht gebaut.

Gebaut wurde im September 2026, mit KI-Agenten als Werkzeug und einem Entwicklungslog, das je Sitzung festhält, was gebaut wurde, was nicht funktionierte und was die Tests gefunden haben. Das Log ist Teil der Abgabe, mit den Fehlern.

Meine Rolle

- **Fachrecherche:** Dokumententypologie, Pflichtmatrix je Incoterm, Regelkatalog mit 52 Regeln und Rechtsverweisen, Prozess der Nachforderung, Regulatorik bis 2028; jede Rechtsangabe als Sekundärrecherche gekennzeichnet.
- **Architektur:** Akte statt Dokument, Assertions und Fakten getrennt, Regeln als Daten, n8n als Orchestrierung mit Code-Nodes als Build-Artefakt; elf ADRs im Moment der Entscheidung.
- **Workflows:** sieben n8n-Workflows, ein eigener n8n-Node in TypeScript, Fehlerpfad mit Wiedervorlage, Zeitplan und Mail in beide Richtungen.
- **Extraktion:** Python-Dienst mit Textlayer und OCR, labelgetriebene Feldextraktion für fünf Belegtypen, UN/CEFACT CII in beide Richtungen, zwei IDP-Anbieter hinter einer Nahtstelle, Modellvorschlag mit Pseudonymisierung.
- **Regelwerk:** 14 ausführbare Regeln in JavaScript, Konfidenzpfad, Übersteuerung, Nachforderung mit Fristen; ein Gate verbietet nackte Zahlen im Regelcode.
- **Oberfläche:** Angular mit ngRx hinter nginx, Kontrast nachgerechnet, axe über jede Ansicht, Übersicht aller Akten.
- **Betrieb und Doku:** Compose mit zehn Containern, Prometheus mit elf Alarmregeln und Runbook je Alarm, Rauchtest in neun Runden, Prozesslandschaft als BPMN, Systemlandschaft, Betriebshandbuch.

3 Funktionsumfang

- **Eingang:** Akte als JSON aus einem Vorsystem, Belege als PDF über Formular oder Webhook, Rechnung als CII-XML, Antwort per E-Mail mit Anhang.
- **Extraktion:** Textlayer mit Koordinaten, sonst Tesseract mit Wortkonfidenz; Klassifikation in fünf Belegtypen plus Ursprungserklärung; jede Assertion mit Seite, Box, Konfidenz und Methode. Alternativ Google Document AI oder Azure Document Intelligence als Leser.
- **Prüfung:** Pflichtmatrix nach Incoterm, 14 Regeln zu Mengen, Werten, Warennummern, Container, Ursprung, Präferenz und Draft-Status; Ergebnis je Regel `erfüllt`, `verletzt` oder `nicht_pruefbar`; ein Prüfziffernfehler bei niedriger Konfidenz ist ein Lesefehler, keine Ablehnung.
- **Übersteuern:** Ein Mensch stellt seine Verantwortung neben einen Befund, mit Name aus der Anmeldung und Begründung; der Befund bleibt stehen, die Übersteuerung verfällt beim nächsten Regelstand.
- **Nachforderung:** Fehlende Nachweise werden zum Vorgang mit Frist relativ zum Zoll-Cut-off, versendet per Mail, erinnert und eskaliert; die Antwort landet über IMAP wieder in der Akte und stößt die Prüfung neu an.
- **Oberfläche:** Belege einreichen, Befund mit Fundstelle lesen, übersteuern; Übersicht mit Entscheidung, offenen Nachforderungen und unzugeordneter Post.
- **Betrieb:** Fehlerpfad mit Antwort 500 und Wiedervorlage, Wiederholung gescheiterter Läufe, Zähler je Workflow und Regel, elf Alarmer mit Runbook, Grafana-Dashboard aus dem Repo.

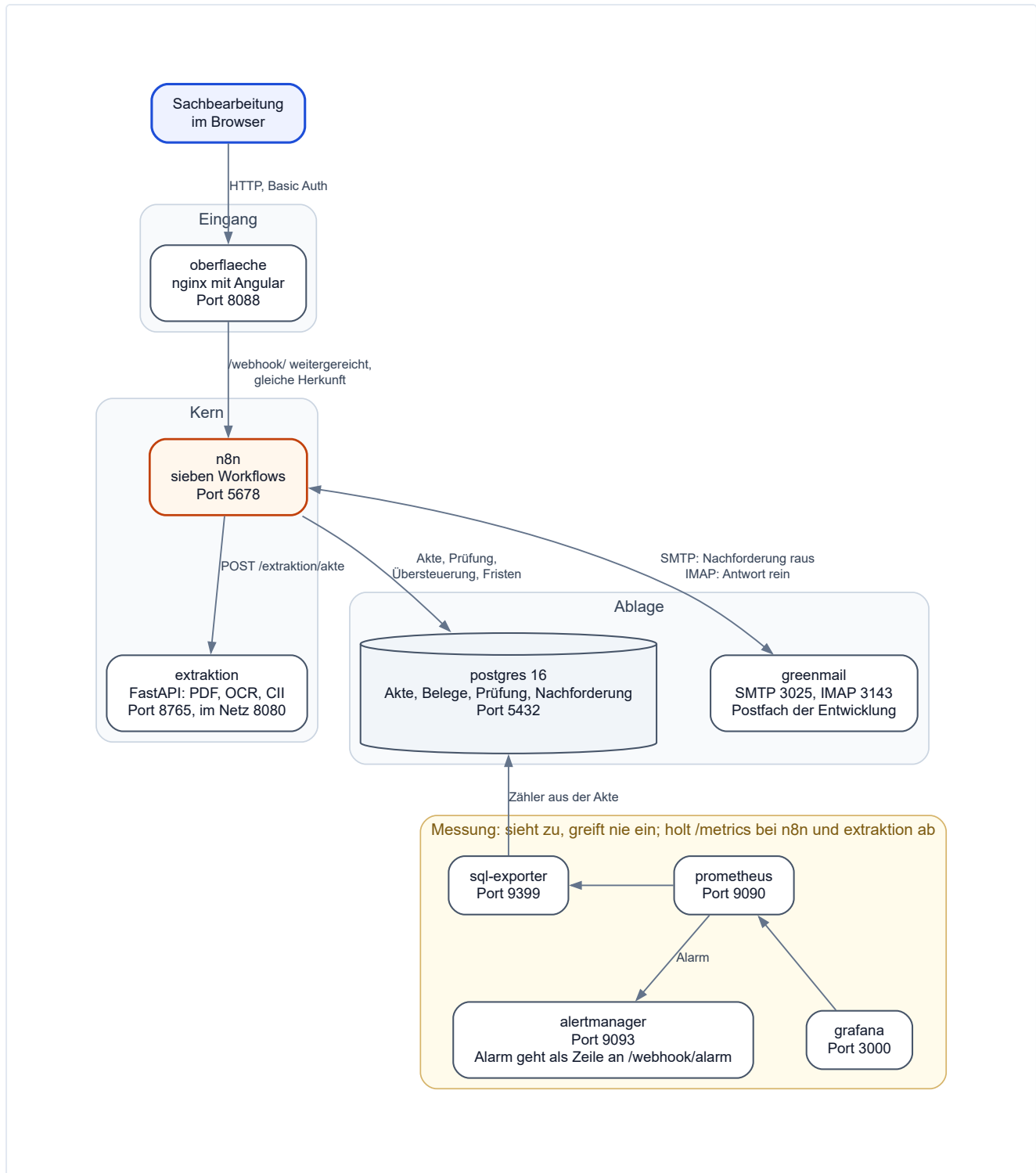
4 Architektur

Bereich	Technologie
Orchestrierung	n8n 1.114 in Docker, sieben Workflows als JSON-Export, Code-Nodes gebündelt aus <code>src/</code> und dem Regelkatalog, ein eigener Node <code>zollPilotExtraktion</code> in TypeScript, Import beim Start statt Klicks im Editor
Entscheidung	JavaScript ohne Framework in <code>src/</code> : Aktenaufbau, Pflichtmatrix, 14 Regeln, Übersteuerung, Nachforderung; Schwellen und Rechtsverweise in <code>rules.yaml</code> mit Gültigkeitsdatum
Extraktion	Python 3.12, FastAPI, pdfplumber, pypdfium2, Tesseract, lxml für CII gegen das D16B-Schema; Anbieterleser für Google Document AI und Azure über REST mit Aufzeichnungen im Repo
Oberfläche	Angular 22 mit ngrx, ausgeliefert von nginx, das <code>/webhook/</code> an n8n weiterreicht (gleiche Herkunft statt CORS) und den angemeldeten Namen als Kopfzeile anhängt
Ablage und Post	Postgres 16, nur anhängend, mit Views für Regelergebnis und Übersteuerungswirkung; GreenMail für SMTP und IMAP in der Entwicklung, ein echtes Postfach über <code>.env</code>
Betrieb und Qualität	Docker Compose mit zehn Diensten und Healthchecks, Prometheus, Alertmanager, SQL-Exporter, Grafana; Git-Hook und GitHub Actions mit acht Jobs; Gates für Belege, Prosa, Verweise, Regeln, Kontrast, Workflows, Diagramme

Schichten

Schicht	Aufgabe
Browser	Angular hinter Basic Auth am Proxy. Reicht Stammdaten und Belege ein, zeigt den Befund, schickt eine Übersteuerung. Entscheidet nichts.
n8n	Nimmt an, reicht weiter, legt ab, verzweigt, antwortet. Entschieden wird ausschließlich im Code-Node „Akte prüfen“ und in zwei Nodes der Nachforderung, alle aus <code>src/</code> gebündelt.
Extraktion	Liest und behauptet: Wörter mit Koordinaten, Belegtyp, Felder, Assertions mit Konfidenz. Setzt keinen Fakt, kennt keine Schwelle, ruft im Entscheidungsweg kein Modell.
<code>src/</code>	Entscheidet deterministisch: Fakten nur aus finalen Belegen, Nachweis statt Dokument, hart vor weich, Lesefehler vor Fachfehler, Übersteuerung verantwortet statt umentscheidet.
Postgres	Akte, Dokumente, Assertions je Prüfung, Prüfung, Übersteuerung, Nachforderung, Versand, Posteingang, Fehler, Wiedervorlage, Alarm. Nur anhängend.
Messung	Prometheus holt Zähler bei n8n, Extraktion und Datenbank; kein Pfeil führt zurück in die Akte.

Systembild



Jeder Port ist an localhost gebunden. Ein zehnter Container importiert beim Start die Workflows und endet; der Editor zeigt, was im Repo steht.

Vier Grundsätze

- **Nachweis statt Dokument.** Geprüft wird „ist der Warenwert nachgewiesen“, nie „liegt eine Rechnung vor“. Deshalb läuft eine Rechnung als PDF, als Scan oder als CII-Datensatz durch dieselben Regeln.
- **Rohwert und Aktenwert getrennt.** Was ein Beleg sagt, wird nie überschrieben. Fakten sind Ableitungen aus finalen Belegen; ein Entwurf des B/L wird gelesen, aber setzt keinen Fakt.
- **Regeln sind Daten.** Schwellen, Toleranzen und Rechtsverweise stehen im Katalog mit Gültigkeitsdatum. Ein Gate lehnt jede nackte Zahl im Regelcode ab.
- **Kein Modell in der Entscheidung.** Modelle lesen, klassifizieren, schlagen vor. Freigabe oder Blockade fällt deterministisch auf normalisierten Werten. Ein Modellvorschlag zum Belegtyp hängt neben dem Dokument, der Typ bleibt `unclassified`.

Workflows

Workflow	Aufgabe
Akte prüfen	Zwei Eingänge, <code>/webhook/akte</code> mit Assertions und <code>/webhook/belege</code> mit PDFs; Extraktion, Prüfung, Ablage, Antwort 200 oder 422 mit allen Befunden. Fehlerzweig mit Wiedervorlage und Antwort 500.
Nachforderungen	Täglich oder auf Zuruf; entscheidet je Akte, was neu, erinnert oder eskaliert wird, versendet per SMTP, hält den Versand fest.
Posteingang	Holt Antworten per IMAP, ordnet sie über das Aktenzeichen zu, extrahiert Anhänge, prüft die Akte neu; Unzugeordnetes wird festgehalten, nicht verworfen.
Übersicht lesen	<code>GET /webhook/akten</code> für den Übersichtsbildschirm. Eine Abfrage, kein Urteil.
Lauf wiederholen	Schickt einen gescheiterten Lauf aus der Wiedervorlage noch einmal an denselben Eingang.
Alarm	Nimmt Alarmer vom Alertmanager an und legt sie als Zeile mit Beginn und Ende ab.
Fehler	Der Auffangworkflow von n8n: reduziert einen Abbruch auf das Aussagekräftige ohne Belegtext und legt ihn ab.

5 Technische Kernprobleme

Sechs Stellen, an denen das Projekt über „Workflow ruft Dienst“ hinausging.

ENTSCHEIDUNG

Ein Lesefehler ist kein Fachfehler

Problem

Eine falsch gelesene Containernummer fällt durch die Prüfziffer. Wer daraus eine fachliche Ablehnung macht, blockiert eine korrekte Sendung wegen eines schlechten Scans; wer sie durchwinkt, hat die Prüfziffer umsonst.

Lösung

Jede Assertion trägt eine Konfidenz, das Minimum über ihre Wörter. Fällt ein Prüfziffern- oder Summenvergleich durch und liegt die Konfidenz unter der Katalogschwelle, ist das Ergebnis `re_extraction_required`, nicht `verletzt`. Zehn von vierzehn Regeln kennen diesen Pfad; der Rauchtest führt ihn mit einem echten, absichtlich schlechten Scan vor.

Warum es zählt

Die Grenze zwischen Lesen und Entscheiden ist die wichtigste Linie im System. Die Extraktion kennt die Schwelle nicht, das Regelwerk kennt keine Bilder, und trotzdem greifen beide an derselben Stelle ineinander.

DATENMODELL

Die Akte statt des Dokuments

Problem

Zwei Belege widersprechen sich, ein B/L liegt erst als Entwurf vor, eine Rechnung kommt zweimal. Ein Dokumentenmodell muss dann entscheiden, welches gewinnt, und verliert dabei, was das andere sagte.

Lösung

Belege liefern Assertions mit Fundstelle und Konfidenz, die nie überschrieben werden. Fakten entstehen nur aus finalen Belegen; ein Entwurf bleibt sichtbar, setzt aber nichts. Die Pflichtmatrix fragt nach Nachweisen, nicht nach Dokumenten, und nennt je Nachweis, wer ihn liefern muss. Damit läuft eine Rechnung als XML-Datensatz mit Konfidenz 1 und ohne Fundstelle durch dieselben Regeln wie ein Scan.

Warum es zählt

Eine Nachforderung braucht Feld, Grund und Adressat. Das geht nur, wenn das System weiß, welcher Nachweis fehlt und wer ihn schuldet, und nicht bloß, dass ein PDF nicht da ist.

NAHTSTELLE

Zwei IDP-Anbieter, ein Feldextraktor

Problem

Das Anforderungsprofil verlangt Erfahrung mit gängigen IDP-Werkzeugen. Ein Anbieter, der fertige Feldwerte liefert, würde die eigene Extraktion ersetzen und den Vergleich wertlos machen. Gebraucht wurde ein Anbieter genau an der Schicht „Lesen“, mit Wörtern, Koordinaten und Konfidenz.

Lösung

Google Document AI und Azure Document Intelligence liefern beide Wörter mit Umriss. Je Anbieter eine Übersetzungsfunktion in dieselbe Form wie der lokale Leser, Antworten einmal aufgezeichnet und im Repo, Tests und Bewertung ohne Zugang. Der erste Lauf lag bei Azure bei 26,8 Prozent, bei Google bei 86,3 Prozent. Beide Male steckte der Fehler in der Übersetzung: Zeilen sind bei den Anbietern Zellen, Satzzeichen und Bindestriche werden abgetrennt, und ein schiefer Scan lässt die Oberkante mit dem Abstand wandern. Korrigiert wurde je eine Stelle im Anbietermodul. Kein Feldextraktor, keine Regel, kein Pfad wurde angefasst. Ergebnis: alle drei Leser 570 von 570 Feldern, 8 von 8 Entscheidungen.

Warum es zählt

Ein Anbieter hinter einer Nahtstelle kann Zufall sein. Zwei Anbieter mit gegensätzlichen Eigenheiten, die beide nur eine Übersetzung brauchten, sind ein Beleg. Und der Weg dorthin zeigt eine Lehre, die zweimal gebraucht wurde: Ein Schemabeispiel aus dem Gedächtnis prüft die eigene Vorstellung, nicht die Wirklichkeit. Erst die echte Antwort gegen das Golden Set fand die Fehler.

VERANTWORTUNG

Übersteuern, ohne umzuentscheiden

Problem

Ein Befund ist fachlich richtig, aber die Sendung muss trotzdem raus. Wer den Befund ändert, verliert die Spur. Wer nichts ändern kann, hat kein Werkzeug.

Lösung

Eine Übersteuerung ist ein eigener Datensatz mit Name aus der Anmeldung am Proxy, Begründung und der Fassung des Regelkatalogs. Das Regelergebnis bleibt stehen; daneben entsteht `freigabe_nach_override`, und der HTTP-Status folgt dieser zweiten Spalte. Ändert sich der Katalog, ist die Übersteuerung verbraucht und muss neu verantwortet werden.

Warum es zählt

Die Frage „wer hat das entschieden, und auf welcher Grundlage“ hat immer eine Antwort. Ein Audit sieht beides: was das System sagte und wer es überstimmt hat.

BETRIEB

Ein gescheiterter Lauf ist keine Freigabe

Problem

Fällt der Extraktionsdienst aus, bricht der Workflow ab. n8n zählt einen abgefangenen Fehler als erfolgreichen Lauf, und der Aufrufer bekommt entweder nichts oder eine leere Antwort, die wie ein Ergebnis aussieht.

Lösung

Jeder kritische Node hat einen Fehlerausgang, der den Fehler in eine eigene Tabelle schreibt, eine Wiedervorlage ablegt und mit 500 samt Ausführungsnummer antwortet. Ein eigener Webhook wiederholt den Lauf aus der Wiedervorlage. Die Alarmregel zählt nicht Erfolge, sondern das Wachstum der Fehlertabelle, weil die Erfolgszähler von n8n hier lügen würden.

Warum es zählt

Monitoring, das die falsche Zahl anschaut, ist schlimmer als keines. Der Rauchtest hat zwei eigene Runden dafür: Eine kaputte Akte muss 500 geben und in der Fehlertabelle stehen; dann wird der Extraktionsdienst angehalten, die Antwort muss 500 mit `wiederholbar` sein, und die Wiederholung nach dem Neustart muss dieselbe Entscheidung liefern wie ein normaler Lauf.

DETAILS

Was man erst im Betrieb merkt

Problem und Lösung

Fünf Punkte, die kein Lesen des Codes zeigt, je mit der Behebung:

- **n8n:** `$input.first()` liefert in einer Verzweigung `undefined`, nicht ein leeres Objekt; der Fehlerausgang eines Nodes ist Ausgang 1, nicht 0. Beides steht jetzt als Hausstil im Repo.
- **Ratenlimit:** Die kostenlose Stufe von Azure erlaubt 20 Aufrufe pro Minute; die Aufzeichnung lief schneller. Lösung: Backoff nach `Retry-After`, kein Beleg geht verloren.
- **Shell:** `curl | grep -q` unter `pipefail` bricht auf dem CI-Läufer ab, lokal nie. Lösung: erst einfangen, dann suchen.
- **Browser:** Chromium auf Linux rendert sieben Pixel höher als auf Windows; die Aktenspalte bekam in der CI einen Rollbalken. Lösung: Kopf und Fuß je vier Pixel schmaler, Test bei 900 Pixel Höhe.
- **Windows:** Die Git-Bash schreibt `/repo/...` in Docker-Aufrufen zu einem Windows-Pfad um. Lösung: `MSYS_NO_PATHCONV=1`, im Betriebshandbuch notiert.

Warum es zählt

Keiner dieser Punkte steht in einer Dokumentation der Werkzeuge. Man findet sie, indem man den Stack wirklich hochfährt, die CI wirklich rot werden lässt und den Fehler bis zur Ursache verfolgt. Das ist der Unterschied zwischen fertig gebaut und fertig geprüft.

6 Datenmodell

Zwölf Tabellen, alle nur anhängend. Nichts wird überschrieben, eine neue Prüfung ist eine neue Zeile.

Tabelle	Zweck und Besonderheit
shipment , document	Die Sendung mit Stammdaten und Fristen; jeder Beleg mit Hash, Typ, Status (Entwurf oder final) und Aussteller. Ein unbekannter Beleg hängt als unclassified mit Hinweis daran, wird nie verworfen.
document_field_assertion	Was ein Beleg sagt: Pfad, Wert, Rohwert, Konfidenz, Seite, Box, Methode. Nie überschrieben, je Prüfung neu angehängt.
canonical_fact	Der Aktenwert je Pfad, abgeleitet nur aus finalen Belegen, mit Herkunft.
pruefung	Entscheidung, Befunde je Regel, Pflichtbefunde, Nachforderungen, Freigabe und Freigabe nach Übersteuerung, Katalogfassung. Quelle der fachlichen Zähler im Monitoring.
override	Wer welchen Befund mit welcher Begründung übersteuert hat, gebunden an die Katalogfassung; die View override_wirkung zeigt, ob eine Übersteuerung noch gilt oder verbraucht ist.
request_case , request_versand	Die Nachforderung als Vorgang: Feld, Grund, Adressat, Frist, Stufe (neu, erinnert, eskaliert), und jeder Versand mit Zeitpunkt und Empfänger.
mail_eingang	Jede eingegangene Mail mit Zuordnung zur Akte oder dem Grund, warum keine möglich war.
workflow_fehler , wiedervorlage	Ein gescheiterter Lauf mit Node, Fehler und Ausführungsnummer, ohne Belegtext; daneben, was erneut hineinmüsste, damit er wiederholt werden kann.
alarm	Jeder Alarm vom Alertmanager als Zeile mit Beginn und Ende.

7 Qualität und offene Punkte

Maßnahme	Was sie leistet
Gates im Hook und in der CI	Beleg-Check: jede Behauptung über die Arbeitsweise hat einen Pfad im Repo, jede ADR ihre Pflichtabschnitte. Prosa-Check gegen ausgeschriebene Umlaute. Verweis-Check gegen tote Links. Regel-Check: keine nackte Zahl im Regelcode, jede Regel mit Test und Katalogeintrag. Workflow-Check: Struktur, keine Geheimnisse, Bild je Workflow. Diagramm-Check: Bild passt zur Quelle.
Tests in vier Sprachen	141 in JavaScript (Regeln, Validatoren, Akte, alle Testakten gegen ihre Erwartung), 147 in Python (Extraktion, OCR, CII, Anbieter, Modellvorschlag), 92 Unit- und 23 Ende-zu-Ende-Tests in Angular mit axe über jede Ansicht, 12 für den eigenen n8n-Node ohne n8n.
Golden Set und Basislinie	Acht synthetische Akten mit 32 Belegen, byteidentisch reproduzierbar; Field Exact Match je Belegtyp und Entscheidung je Akte gegen eine bewusst geschriebene Basislinie. Ein Lauf darunter ist rot.
Rauchtest gegen den Stack	Neun Runden: Akten, PDFs, Proxy, Übersteuerung, gescheiterter Lauf, Wiederholung, Monitoring, Nachforderung, Posteingang. Läuft im CI-Job <code>betrieb nach docker compose up</code> .
Datenschutz	Nur synthetische Belege; kein Belegtext im Log; jeder Modell- und Anbieteraufruf nur aus Aufzeichnungen, Zugänge nur aus der Umgebung; Pseudonymisierung vor jedem Modellaufruf.
Entwicklungslog	Je Sitzung: was gebaut wurde, was nicht funktionierte, was die Tests gefunden haben. Mit den Fehlern, nicht ohne.

Was die Vergleichsläufe gefunden haben

Beide IDP-Anbieter lagen im ersten Lauf unter der Basislinie, Azure bei 26,8 Prozent, Google bei 86,3 Prozent, und beide Male war die eigene Übersetzung schuld, nicht der Anbieter. Alle Tests waren vorher grün, weil sie dieselbe Annahme trugen wie der Code. Dass es hier steht, ist Absicht: Der Wert eines Vergleichslaufs liegt darin, dass er die eigene Vorstellung gegen die Wirklichkeit hält.

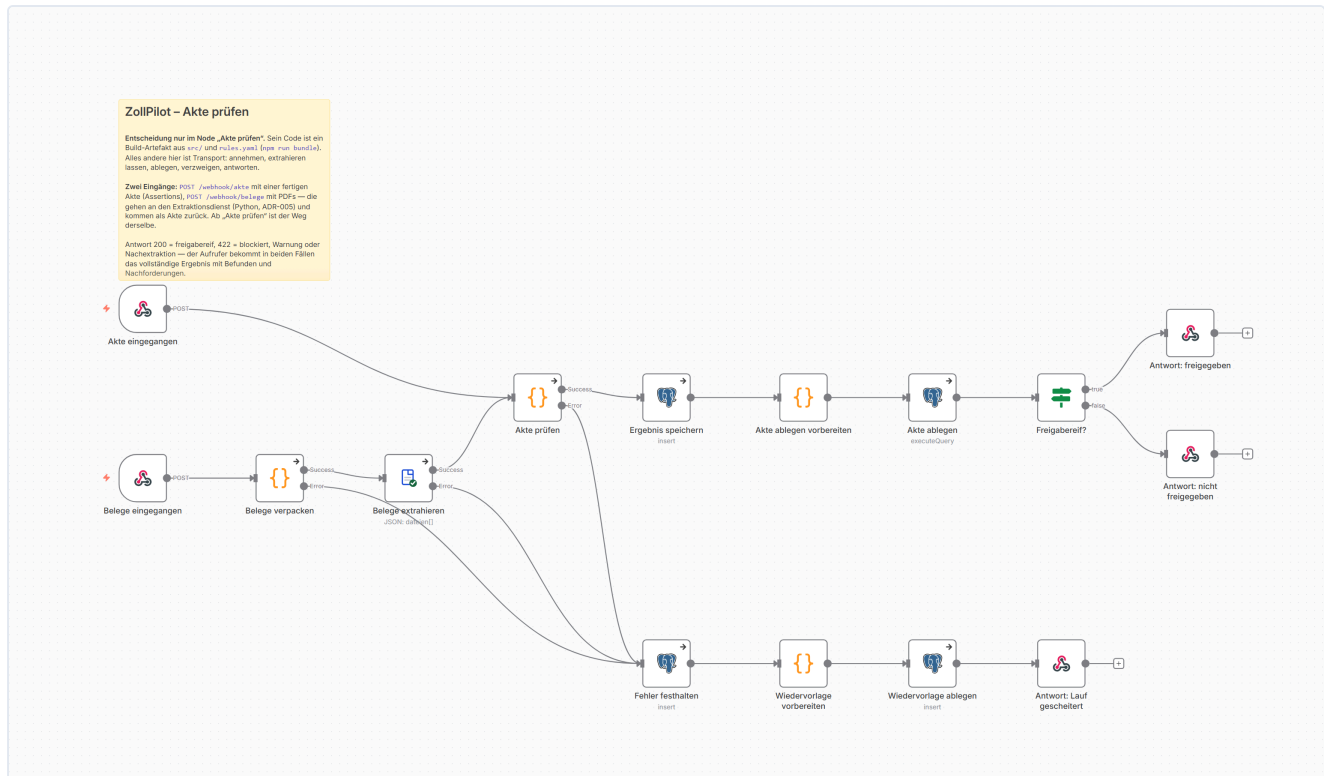
Offen, und so benannt

- **Jahre im Betrieb.** Das Repo zeigt, wie n8n produktionsstauglich eingesetzt wird; es kann nicht ersetzen, einen Betrieb über Monate getragen zu haben. Das steht so im Anforderungsmapping.
- **Echte Belege.** Alle Belege sind synthetisch, eine Layoutfamilie, ein Scan. Wie gut die Extraktion auf fremden Rechnungen liest, ist hier nicht messbar; die Regeln sagen dann `nicht_pruefbar`.
- **DMS, ERP, Kubernetes.** Die Ablage ist Postgres, der Betrieb ein Compose auf einer Maschine. Die Stellen dafür sind benannt, gebaut sind sie nicht.
- **Rechtsverweise.** Alle aus Sekundärrecherche; keine Regel trägt `verified`. Vor einem echten Einsatz prüft das ein Zollfachmensch.

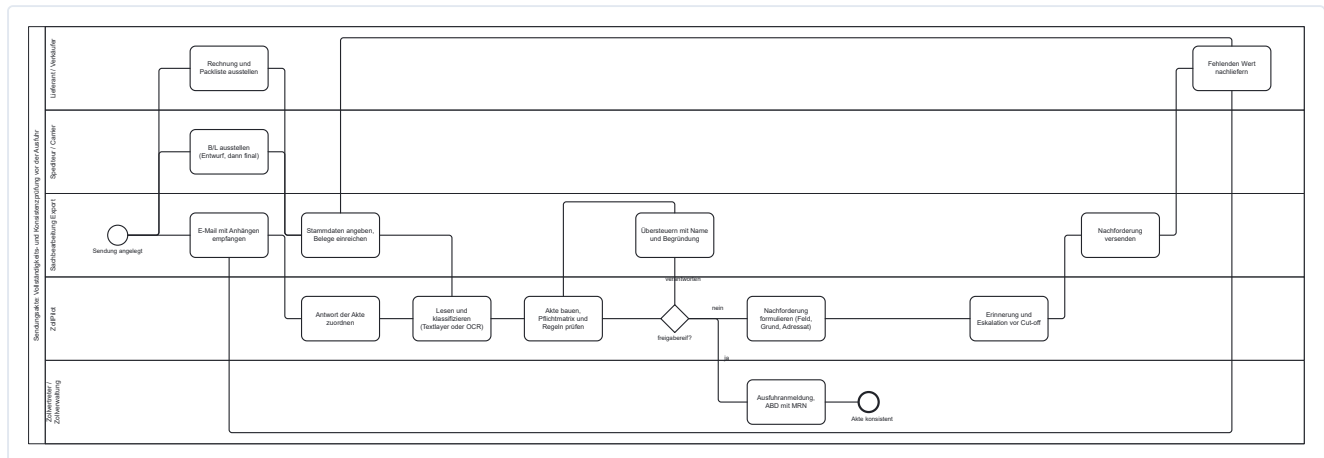
Stand

Der Stack läuft lokal mit einem Befehl, die CI ist grün, alle Punkte des Anforderungsprofils sind im Mapping mit Beleg oder mit dem ehrlichen Vermerk versehen, was ein Prototyp ohne Kundendaten nicht zeigen kann.

8 Einblicke



Der Hauptworkflow im Editor: zwei Eingänge, ein Code-Node, der entscheidet, Ablage, Verzweigung, und darunter der Fehlerzweig mit Wiedervorlage.



Prozesslandschaft als BPMN 2.0, aus der Quelldatei gezeichnet: fünf Rollen, eine Akte, von der angelegten Sendung bis zur Antwort auf eine Nachforderung.

ZOLLPILOT

Prüfakte

Übersicht

Angemeldet als sachbearbeitung

Prüfakte

Belege einer Sendung einreichen. Die Extraktion liest sie, das Regelwerk entscheidet deterministisch, und Sie sehen je Befund Begründung und Adressat.

STAMMDATEN VORBELEGT

Akten-Nummer

ZP-2026-0001

Stichtag

2026-09-12

Richtung

Ausfuhr in ein Drittland

Verkehrsträger

Seefracht

Präferenz

wird beansprucht

Klausel

FOB · Hamburg (DEHAM)

Häfen

DEHAM → SGSIN

Warennummern

84133000, 84842000

Ändern

BELEGE

PDF-Dateien einer Sendung: Handelsrechnung, Packliste, Bill of Lading. Welcher Beleg welcher ist, entscheidet die Extraktion; die Reihenfolge spielt keine Rolle.

Dateien auswählen

... oder Dateien hierher ziehen.

Noch kein Beleg ausgewählt.

Akte einreichen

Gesperrt: Mindestens ein Beleg wird gebraucht.

Noch keine Prüfung

Belege beilegen, dann die Akte einreichen. Das Ergebnis erscheint hier, an dieser Stelle, in dieser Breite.

WAS GEPRÜFT WIRD

Nachweispflichten

aus dem Katalog, je Pflicht der Beleg, der sie erfüllt

Regelwerk

harte und weiche Regeln, mit Konsequenz und Rechtsgrundlage

Nachforderungen

mit Adressat, Kopie und fertigem Anschreiben

Prototyp mit synthetischen Testdaten. Alle Rechtsverweise stammen aus Sekundärrecherche und sind vor produktivem Einsatz zu verifizieren.

Einreichen: Stammdaten der Sendung und die Belege als Dateien, hinter der Anmeldung am Proxy. Der angemeldete Name wandert als Kopfzeile mit und steht später an jeder Übersteuerung. Das Formular kennt keine Regel; es reicht ein und zeigt, was zurückkommt, mit Befund, Fundstelle und Begründung je Regel.

ZOLLPILOT

Prüfakte

Übersicht

Angemeldet als sachbearbeitung

Übersicht

Alle geprüften Akten, ihre offenen Nachforderungen und die Post, die keiner Akte zuzuordnen war. Die Zahlen kommen aus Postgres; entschieden wird hier nichts.

Neu laden

Stand 13.09.2026, 02:37

AKTEN

8

BLOCKIERT

4

FREIGABEREIF

4

OFFENE NACHFORDERUNGEN

0

UNZUGEORDNETE POST

10

Akten 8

AKTE	ENTSCHEIDUNG	ZULETZT GEPRÜFT	BELEGE	OFFENE NACHFORDERUNGEN	LETZTE STUFE	ZOLL-CUT-OFF
ZP-2026-0002	Freigabereif	13.09.2026, 02:26	6	0		15.09.2026, 16:00
ZP-2026-0001	Freigabereif	13.09.2026, 02:26	5	0		15.09.2026, 16:00
ZP-2026-0004	Freigabereif	13.09.2026, 02:26	5	0		15.09.2026, 16:00
ZP-2026-0003	Blockiert	13.09.2026, 02:26	5	0		15.09.2026, 16:00
ZP-2026-0006	Freigabereif	13.09.2026, 02:26	5	0		15.09.2026, 16:00
ZP-2026-0007	Blockiert	13.09.2026, 02:26	5	0		15.09.2026, 16:00
ZP-2026-0005	Blockiert	13.09.2026, 02:26	5	0		15.09.2026, 16:00
ZP-2026-0008	Blockiert	13.09.2026, 02:26	5	0		15.09.2026, 16:00

Unzugeordnete Post 10

EINGANG	VON	BETREFF	GRUND	ANHÄNGE
13.09.2026, 02:26	Unbekannt <jemand@example.test>	Frage ohne Aktennummer	keine Aktennummer in Betreff oder Text	0

Prototyp mit synthetischen Testdaten. Alle Rechtsverweise stammen aus Sekundärrecherche und sind vor produktivem Einsatz zu verifizieren.

Übersicht: jede geprüfte Akte mit Entscheidung, Zahl der Belege, offenen Nachforderungen, letzter Stufe und Zoll-Cut-off, darunter die Post, die keiner Akte zuzuordnen war, mit dem Grund. Die Zahlen kommen aus Postgres; entschieden wird auf diesem Bildschirm nichts, und die Fußzeile sagt, dass alle Daten synthetisch sind.

Gabriel Suter · kontakt@gabrielsuter.dev

13 / 14

9 Was das Projekt zeigt

Fähigkeit	Wo sie sichtbar wird
n8n in Produktion gedacht	Entscheidung außerhalb des Workflows, Code-Node als Build-Artefakt, eigener Node, Fehlerpfad mit Wiedervorlage, Import beim Start, Betriebshandbuch mit Runbook je Alarm
IDP und OCR	Textlayer und Tesseract mit Koordinaten, fünf Belegtypen, CII in beide Richtungen, zwei Anbieter hinter einer Nahtstelle, Golden Set mit Basislinie
Zollfachlichkeit	Dokumententypologie, Pflichtmatrix je Incoterm, 52 Regeln im Katalog mit Rechtsgrundlage, Ausfuhrbegleitdokument mit MRN, Präferenznachweis, Grenzen des eigenen Wissens benannt
Architektur mit Begründung	Elf ADRs im Moment der Entscheidung, jede mit verworfenen Optionen, Nachteilen und dem Abschnitt, wann sie falsch wäre
Prüfen statt annehmen	Sieben Gates, 415 Tests, Rauchtest in neun Runden, Vergleichsläufe gegen echte Anbieter, ein Entwicklungslog mit den Fehlern
Prozessdokumentation	Prozesslandschaft als BPMN, Datenfluss und Systemlandschaft aus Quelldateien gezeichnet, sieben Workflows als Bild, alles mit Gate gegen Veralten
KI als Werkzeug, mit Grenze	Gebaut mit KI-Agenten, jede Sitzung protokolliert; ein Modell schlägt vor und entscheidet nie, hinter Pseudonymisierung und aus Aufzeichnungen
Übergabe	Ein Befehl für den Stack, Betriebshandbuch, Alarmer mit Handgriffen, Anforderungsmapping als Vertrag
Stand	September 2026 · lokal lauffähig mit <code>docker compose up -d --wait</code>
Kontakt	Gabriel Suter · kontakt@gabrielsuter.dev · gabrielsuter.dev
Quellcode	github.com/SuterGabriel/ZollPilot, mit Entwicklungslog und allen Entscheidungen
Auftraggeber	Ein Anforderungsprofil, erfasst in <code>docs/ANFORDERUNGEN.md</code> , Punkt für Punkt mit Beleg